

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет "Львівська політехніка"
Інститут комп'ютерних наук та інформаційних технологій



ОСНОВИ UNITY ТА РОЗРОБКА НА C#

МЕТОДИЧНІ ВКАЗІВКИ

до лабораторної роботи № 2

з курсу: «Технології доповненої реальності»

Львів – 2025

1. МЕТА РОБОТИ

Unity є основним рушієм для AR-додатків на базі AR Foundation. Перш ніж розміщувати віртуальні об'єкти в реальному середовищі, необхідно впевнено володіти базовими інструментами Unity: скриптами, трансформаціями та фізикою. Саме ці навички стануть фундаментом для наступних лабораторних робіт, де ви працюватимете з розпізнаванням площин, якорями та взаємодією з AR-сценою.

Після виконання лабораторної роботи студент зможе:

- орієнтуватися в архітектурі Unity (GameObject, Component);
- пояснити життєвий цикл MonoBehaviour (Awake → Start → Update);
- створювати та налаштовувати C# скрипти в Unity;
- використовувати Transform для переміщення, обертання та масштабування об'єктів;
- працювати з фізичним рушієм Unity (Rigidbody, Collider, Trigger).

2. ТЕОРЕТИЧНІ ВІДОМОСТІ

2.1. Архітектура Unity: GameObject та Component

Unity побудована на **компонентній архітектурі**. Кожен об'єкт у сцені — це **GameObject**, а його поведінка визначається набором **компонентів (Component)**.

GameObject "Cube"
Component: Transform Position: (0, 0.5, 0) Rotation: (0, 0, 0) Scale: (1, 1, 1)
Component: MeshFilter Mesh: Cube
Component: MeshRenderer Material: Default-Material

```
Component: Spinner (ваш скрипт!)
Speed: 30
Axis: (0, 1, 0)
```

Правило: GameObject без компонентів — порожній контейнер. Саме компоненти визначають, як об'єкт виглядає, поводитьься та взаємодіє.

2.2. Компонент Transform

Transform — єдиний обов'язковий компонент кожного GameObject. Він визначає:

Властивість	Тип	Опис
position	Vector3	Позиція у світових координатах (x, y, z)
localPosition	Vector3	Позиція відносно батьківського об'єкта
rotation	Quaternion	Орієнтація у 3D-просторі
eulerAngles	Vector3	Орієнтація у градусах (зручніше для людини)
localScale	Vector3	Масштаб (1, 1, 1 = оригінальний розмір)

Корисні методи Transform:

```
// Переміщення
transform.Translate(Vector3.forward * speed * Time.deltaTime);

// Обертання
transform.Rotate(Vector3.up * speed * Time.deltaTime);

// Поворот до цілі
transform.LookAt(targetTransform);
```

2.3. Життєвий цикл MonoBehaviour

MonoBehaviour — базовий клас для всіх Unity-скриптів. Його методи викликаються автоматично:

ІНІЦІАЛІЗАЦІЯ	
Awake()	← Викликається першим (один раз)
↓	
OnEnable()	← Коли об'єкт стає активним
↓	
Start()	← Перед першим Update (один раз)
ІГРОВИЙ ЦИКЛ (кожен кадр)	
FixedUpdate()	← Фізика (фіксований крок, 50 Гц)
↓	
Update()	← Логіка (кожен кадр, частота варіює)
↓	
LateUpdate()	← Після Update (камера, UI)
ЗАВЕРШЕННЯ	
OnDisable()	← Коли об'єкт деактивується
↓	
OnDestroy()	← Коли об'єкт знищується

Коли що використовувати:

Метод	Коли використовувати	Приклад
Awake()	Ініціалізація посилань	<code>rb = GetComponent<Rigidbody>();</code>
Start()	Початкові налаштування	<code>speed = 30f;</code>
Update()	Введення, логіка	<code>if (Input.GetKey(...))</code>
FixedUpdate()	Фізика	<code>rb.AddForce(...)</code>
LateUpdate()	Слідування камери	<code>cam.position = target.position;</code>

2.4. Time.deltaTime

`Time.deltaTime` — час у секундах між поточним і попереднім кадром.

Навіщо? Щоб рух був однаковий на різних FPS:

```
// ❌ НЕПРАВИЛЬНО: швидкість залежить від FPS
transform.Translate(Vector3.forward * speed);
```

```
//  ПРАВИЛЬНО: швидкість однакова на будь-якому FPS
transform.Translate(Vector3.forward * speed * Time.deltaTime);
```

FPS	deltaTime	Без deltaTime (units/сек)	З deltaTime (units/сек)
30	≈0.033	$30 \times 30 = 900$	$30 \times 0.033 \times 30 \approx 30$
60	≈0.017	$30 \times 60 = 1800$	$30 \times 0.017 \times 60 \approx 30$
120	≈0.008	$30 \times 120 = 3600$	$30 \times 0.008 \times 120 \approx 30$

При `speed = 30` без `deltaTime` об'єкт рухається **вдвічі швидше** при вдвічі більшому FPS. З `deltaTime` — завжди **30 units/сек**, незалежно від FPS.

2.5. Фізичний рушій Unity

Unity використовує **PhysX** (від NVIDIA) для симуляції фізики:

Компонент	Призначення
Rigidbody	Додає масу, гравітацію, сили до об'єкта
Collider	Визначає форму для зіткнень (Box, Sphere, Capsule, Mesh)
Trigger	Collider з <code>Is Trigger = true</code> — не блокує, але реєструє перетин

Rigidbody — режими сил:

ForceMode	Опис	Використання
Force	Постійна сила (враховує масу)	Двигун, вітер
Impulse	Миттєвий поштовх (враховує масу)	Стрибок, удар
Acceleration	Прискорення (ігнорує масу)	Гравітація
VelocityChange	Миттєва зміна швидкості	Телепортація руху

2.6. Collider vs Trigger

```
COLLIDER (Is Trigger = false)
```

```
OnCollisionEnter(Collision) — зіткнення почалось
```

```
OnCollisionStay(Collision) — триває контакт
```

```
OnCollisionExit(Collision) — зіткнення закінчилось
```

→ Об'єкти фізично відштовхуються

```
TRIGGER (Is Trigger = true)
```

```
OnTriggerEnter(Collider) — об'єкт увійшов
```

```
OnTriggerStay(Collider) — об'єкт всередині
```

```
OnTriggerExit(Collider) — об'єкт вийшов
```

→ Об'єкти проходять один крізь одного

3. ОБЛАДНАННЯ ТА ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ

Компонент	Вимоги
Unity	2022.3 LTS (встановлено у Лаб 1)
IDE	Visual Studio або VS Code з C# extension
Смартфон	Android 8.0+ / iOS 13+ (для збірки на пристрій) або XR Simulation

Передумова: Лабораторна робота №1 повинна бути виконана. Unity має бути встановлено та налаштовано для Android або iOS.

4. ХІД РОБОТИ

4.1. Частина 1: Створення проекту та сцени (15 хв)

Крок 1.1: Новий проект

1. Відкрийте **Unity Hub** → **New project**
2. Виберіть шаблон **Universal 3D**
3. Назва: **AR_Lab02_UnityBasics**
4. Натисніть **Create project**
5. Дочекайтеся відкриття редактора
6. **Edit** → **Project Settings** → **Player** → **Other Settings** → **Active Input Handling** → змініть на **Both** (Unity перезапуститься)

У Unity 6 цей пункт може бути в **Player Settings** профілю збірки (**File** → **Build Profiles** → **Player Settings**).

Якщо в **Console** бачите `InvalidOperationException: ... switched active Input handling to Input System package` — це саме через це налаштування.

Крок 1.2: Створення структури папок

У вікні **Project (Assets)** створіть папки:

7. ПКМ на **Assets** → **Create** → **Folder** → `Scripts`
8. Повторіть для: `Materials`, `Scenes`

```
Assets/  
├─ Materials/    # Матеріали та текстури  
├─ Scenes/      # Файли сцен  
└─ Scripts/     # C# скрипти
```

Крок 1.3: Побудова сцени

Важливо: Протягом лабораторної ви будете додавати нові об'єкти до сцени. Нічого не видаляйте — наприкінці у вас буде одна сцена з усіма об'єктами.

9. Збережіть поточну сцену: **File** → **Save As** → `Assets/Scenes/Lab02Scene`
10. Додайте **Plane** (підлога): **Hierarchy** → ПКМ → **3D Object** → **Plane**

- Position: (0, 0, 0)
- Scale: (3, 1, 3)

11. Додайте **Cube**: **Hierarchy** → ПКМ → **3D Object** → **Cube**

- Position: (-4, 0.5, 0)

12. Додайте **Sphere**: **3D Object** → **Sphere**

- Position: (-1.5, 0.5, 0)

13. Додайте **Capsule**: **3D Object** → **Capsule**

- Position: (1.5, 0.5, 0)

14. Додайте **Cylinder**: **3D Object** → **Cylinder**

- Position: (4, 0.5, 0)

Крок 1.4: Створення матеріалів

15. У папці **Assets/Materials** створіть чотири матеріали:

- `Mat_Red` — Base Color: червоний
- `Mat_Blue` — Base Color: синій
- `Mat_Yellow` — Base Color: жовтий
- `Mat_Green` — Base Color: зелений

16. Перетягніть матеріали на відповідні об'єкти:

- `Cube` → `Mat_Red`
- `Sphere` → `Mat_Blue`
- `Capsule` → `Mat_Yellow`
- `Cylinder` → `Mat_Green`

Перевірка: У Scene View ви бачите 4 кольорові об'єкти на площині.

4.2. Частина 2: Перший скрипт — Spinner (25 хв)

Крок 2.1: Створення скрипта

17. У папці **Assets/Scripts** → ПКМ → **Create** → **MonoBehaviour Script** (у старіших версіях — **C# Script**)
18. Назвіть `Spinner` (саме так, з великої літери, без пробілів)
19. Двічі клікніть на скрипт — відкриється Visual Studio

Важливо: Назва файлу **ПОВИННА** збігатися з назвою класу. Якщо файл `Spinner.cs`, клас має бути `public class Spinner`.

Крок 2.2: Базовий код Spinner

Замініть весь вміст файлу на:

```
using UnityEngine;

public class Spinner : MonoBehaviour
{
    [Header("Налаштування обертання")]
    [Tooltip("Вісь обертання")]
    public Vector3 rotationAxis = Vector3.up;

    [Tooltip("Швидкість обертання (градуси/секунду)")]
    [Range(0f, 360f)]
    public float speed = 30f;

    void Update()
```



```

    {
        // Обертаємо об'єкт навколо заданої осі
        transform.Rotate(rotationAxis * speed * Time.deltaTime);
    }
}

```

Пояснення коду:

Рядок	Пояснення
<code>using UnityEngine;</code>	Підключає бібліотеку Unity
<code>public class Spinner : MonoBehaviour</code>	Клас-компонент, який можна додати до GameObject
<code>[Header("...")]</code>	Заголовок у Inspector для зручності
<code>[Range(0f, 360f)]</code>	Обмежує значення повзунком у Inspector
<code>public Vector3 rotationAxis</code>	Публічне поле — видиме та редаговане в Inspector
<code>void Update()</code>	Викликається кожен кадр
<code>transform.Rotate(...)</code>	Обертає об'єкт
<code>Time.deltaTime</code>	Компенсація різної частоти кадрів

Крок 2.3: Додавання скрипта до об'єкта

20. Збережіть скрипт (Ctrl+S у Visual Studio)
21. Поверніться до Unity
22. Виберіть **Cube** у Hierarchy
23. В Inspector натисніть **Add Component** → введіть `Spinner` → оберіть **Spinner** зі списку
- Або: перетягніть скрипт `Spinner` з вікна Project на Inspector

У Inspector з'явиться компонент:

```

Spinner (Script)
Налаштування обертання
Rotation Axis    X[0] Y[1] Z[0]
Speed            [====|====] 30

```

Крок 2.4: Тестування

Console: Щоб бачити повідомлення `Debug.Log` — відкрийте **Window** → **General** → **Console**.

24. Натисніть **Play** (▶)

25. Куб має обертатися навколо осі Y

26. **Не зупиняючи Play**, змініть:

- Speed → 120 (швидше)
- Rotation Axis → X: 1, Y: 0, Z: 0 (обертання навколо X)

27. Спостерігайте зміни в реальному часі

28. Натисніть **Play** ще раз для зупинки

Увага: Зміни параметрів під час Play Mode **не зберігаються!** Це зручно для експериментів, але не забудьте повторити потрібні зміни після зупинки.

Крок 2.5: Додавання скрипта до інших об'єктів

29. Виберіть **Sphere** → Add Component → `Spinner`

- Rotation Axis: (1, 0, 0), Speed: 45

30. Виберіть **Capsule** → Add Component → `Spinner`

- Rotation Axis: (1, 0, 1), Speed: 50

31. Виберіть **Cylinder** → Add Component → `Spinner`

- Rotation Axis: (0, 0, 1), Speed: 60

32. Натисніть **Play** — всі 4 об'єкти обертаються з різними параметрами

4.3. Частина 3: Фізика — Rigidbody та сили (25 хв)

Крок 3.1: Підготовка сцени для фізики

33. Додайте новий **Cube**: Hierarchy → ПКМ → 3D Object → Cube

- Назва: `PhysicsCube`
- Position: (0, 3, -3) — окремо від `Spinner`-об'єктів
- Додайте матеріал `Mat_Red`

34. Виберіть **PhysicsCube** → Inspector → Add Component → **Rigidbody**

- Mass: 1
- Use Gravity: 

35. Натисніть **Play** — куб впаде на площину під дією гравітації

Зверніть увагу: Plane вже має Mesh Collider, тому куб зупиняється на ній, а не проходить крізь.

Крок 3.2: Скрипт PhysicsDemo

Створіть новий скрипт `PhysicsDemo.cs` у папці `Scripts`:

```
using UnityEngine;

public class PhysicsDemo : MonoBehaviour
{
    [Header("Налаштування сили")]
    [Tooltip("Величина сили")]
    public float forceAmount = 5f;

    [Tooltip("Напрямок сили")]
    public Vector3 forceDirection = Vector3.up;

    [Tooltip("Тип сили")]
    public ForceMode forceMode = ForceMode.Impulse;

    private Rigidbody rb;

    void Awake()
    {
        // Отримуємо посилання на Rigidbody один раз
        rb = GetComponent<Rigidbody>();

        if (rb == null)
        {
            Debug.LogError($"PhysicsDemo: Rigidbody не знайдено на {gameObject.name}!");
        }
    }

    void Update()
    {
        // У редакторі — пробіл, на телефоні — тап по екрану
        bool activate = Input.GetKeyDown(KeyCode.Space);

        if (!activate && Input.touchCount > 0 && Input.GetTouch(0).phase == TouchPhase.Began)
        {
            activate = true;
        }

        if (activate)
        {
            ApplyForce();
        }
    }
}
```

```
private void ApplyForce()
{
    if (rb == null) return;

    rb.AddForce(forceDirection.normalized * forceAmount, forceMode);
    Debug.Log($"Сила застосована: {forceDirection.normalized *
forceAmount}, режим: {forceMode}");
}
}
```

Пояснення коду:

Елемент	Пояснення
<code>GetComponent<Rigidbody>()</code>	Отримує компонент <code>Rigidbody</code> з цього ж об'єкта
<code>Input.GetKeyDown(KeyCode.Space)</code>	У редакторі — пробіл
<code>Input.GetTouch(0).phase == TouchPhase.Began</code>	На телефоні — тап по екрану
<code>rb.AddForce(...)</code>	Додає фізичну силу до об'єкта
<code>forceDirection.normalized</code>	Нормалізує вектор (довжина = 1)

Крок 3.3: Налаштування

36. Видаліть компонент `Spinner` з **PhysicsCube** (якщо є)

37. Додайте компонент `PhysicsDemo` до **PhysicsCube**

38. Параметри:

- Force Amount: 5
- Force Direction: (0, 1, 0)
- Force Mode: Impulse

Крок 3.4: Тестування фізики

39. Натисніть **Play**

40. Куб впаде на площину

41. Натисніть **Space** (у редакторі) або **тапніть по екрану** (на телефоні) — куб підстрибне

42. Натискайте кілька разів — куб підлітає вище

43. Спробуйте змінити **Force Mode** на `Force` — ефект ледь помітний (`ForceMode.Force` розраховано на постійну дію кожен кадр, а одноразовий тап дає мінімальний імпульс)

44. Змініть **Force Direction** на (1, 1, 0) — куб летить вбік і вгору

Крок 3.5: Експеримент з масою

45. Зупиніть Play Mode
46. Додайте ще один куб: **PhysicsCube2**
 - Position: (2, 3, -3)
 - **Add Component** → **Rigidbody** → Mass: **10**
 - **Add Component** → **PhysicsDemo** → Force Amount: 5
47. Натисніть **Play** → натисніть **Space**
48. **Спостереження:** легший куб (mass=1) підлітає вище, ніж важчий (mass=10) при однаковій силі Impulse

4.4. Частина 4: Тригери та події (25 хв)

Крок 4.1: Підготовка тригер-зони

49. Створіть **Cube**: назва `TriggerZone`
 - Position: (0, 0.5, 5)
 - Scale: (3, 1, 3)
50. У Inspector знайдіть **Box Collider**:
 - Поставте галочку **Is Trigger** 
51. Створіть напівпрозорий матеріал:
 - Новий матеріал `Mat_TriggerZone`
 - Surface Type: **Transparent**
 - Base Color: жовтий з Alpha ~0.3
52. Додайте матеріал до `TriggerZone`

Крок 4.2: Об'єкт для входу в тригер

53. Створіть **Sphere**: назва `TriggerBall`
 - Position: (-3, 0.5, 5) — поруч з `TriggerZone`, але поза нею

- Додайте **Rigidbody** (Use Gravity:  — вимкнути)
- Додайте матеріал `Mat_Blue`

Крок 4.3: Скрипт **TriggerDemo**

Створіть скрипт `TriggerDemo.cs`:

```
using UnityEngine;

public class TriggerDemo : MonoBehaviour
{
    [Header("Ефекти")]
    [Tooltip("Колір при вході в тригер")]
    public Color triggerColor = Color.red;

    [Tooltip("Масштаб при вході в тригер")]
    public float triggerScale = 1.5f;

    private Renderer rend;
    private Color originalColor;
    private Vector3 originalScale;

    void Start()
    {
        rend = GetComponent<Renderer>();
        originalColor = rend.material.color;
        originalScale = transform.localScale;
    }

    void OnTriggerEnter(Collider other)
    {
        // Об'єкт увійшов у тригер-зону
        Debug.Log($"{gameObject.name} увійшов у тригер: {other.gameObject.name}");
        rend.material.color = triggerColor;
        transform.localScale = originalScale * triggerScale;
    }

    void OnTriggerExit(Collider other)
    {
        // Об'єкт вийшов з тригер-зони
        Debug.Log($"{gameObject.name} вийшов з тригера: {other.gameObject.name}");
        rend.material.color = originalColor;
        transform.localScale = originalScale;
    }
}
```

Крок 4.4: Скрипт руху для кулі

Створіть скрипт `SimpleMovement.cs`:

```
using UnityEngine;

public class SimpleMovement : MonoBehaviour
{
    [Tooltip("Швидкість руху")]
    public float moveSpeed = 3f;
}
```

```

void Update()
{
    // Рух стрілками або WASD
    float horizontal = Input.GetAxis("Horizontal"); // A/D або ←/→
    float vertical = Input.GetAxis("Vertical");      // W/S або ↑/↓

    Vector3 movement = new Vector3(horizontal, 0, vertical);
    transform.Translate(movement * moveSpeed * Time.deltaTime);
}
}

```

Крок 4.5: Налаштування та тестування

54. До **TriggerBall** додайте:

- **TriggerDemo** (Trigger Color: Red, Trigger Scale: 1.5)
- **SimpleMovement** (Move Speed: 3)

55. Натисніть **Play**

56. Рухайте кулю клавішами **WASD** — заведіть її в жовту зону

57. При вході — куля стає червоною та збільшується

58. Виведіть кулю назад — повертається до оригіналу

4.5. Частина 5: Збірка на телефон (15 хв)

Крок 5.1: Адаптація введення для телефону

На телефоні немає клавіатури. Замініть `SimpleMovement.cs` на варіант з керуванням акселерометром (працює і на **Android**, і на **iOS**):

```

using UnityEngine;

public class SimpleMovement : MonoBehaviour
{
    public float moveSpeed = 3f;

    // Мінімальний нахил для реакції (щоб уникнути дрейфу)
    private const float DeadZone = 0.05f;

    void Update()
    {
        float horizontal;
        float vertical;

        // На телефоні — акселерометр, у редакторі — клавіатура
        if (!Application.isEditor && SystemInfo.supportsAccelerometer)
        {
            // Тримайте телефон злегка нахиленим від горизонту
            // x = нахил ліворуч/праворуч, y = нахил вперед/назад
            horizontal = Input.acceleration.x;
            vertical = Input.acceleration.y;
        }
    }
}

```

```

        // Dead zone – ігноруємо мінімальний нахил (тремтіння рук)
        if (Mathf.Abs(horizontal) < DeadZone) horizontal = 0f;
        if (Mathf.Abs(vertical) < DeadZone) vertical = 0f;
    }
    else
    {
        horizontal = Input.GetAxis("Horizontal"); // A/D або ←/→
        vertical = Input.GetAxis("Vertical");      // W/S або ↑/↓
    }

    Vector3 movement = new Vector3(horizontal, 0, vertical);
    transform.Translate(movement * moveSpeed * Time.deltaTime);
}
}

```

Важливо: Перевірка `Application.isEditor` гарантує, що в Unity Editor працює клавіатура (WASD), а на телефоні — акселерометр. Dead zone (0.05f) запобігає мимовільному руху від тремтіння рук.

Крок 5.2: Збірка

Unity 6: замість Build Settings використовується **File → Build Profiles** (Ctrl+Shift+B). Створіть новий профіль для потрібної платформи.

Unity 2022: **File → Build Settings →** оберіть платформу **→ Switch Platform.**

Android:

59. Оберіть платформу **Android** (див. примітку вище)
60. Додайте `Lab02Scene` до списку сцен
61. **Player Settings** → Package Name: `com.lpnu.arlab02.petrenko`
62. Підключіть телефон **→ Build And Run**

iOS:

63. Оберіть платформу **iOS** (див. примітку вище)
64. Додайте `Lab02Scene` до списку сцен
65. **Player Settings** → Bundle Identifier: `com.lpnu.arlab02.petrenko`
66. **Build** → відкрийте `.xcodeproj` у **Xcode**
67. У Xcode: **Signing & Capabilities** → Team: ваш Apple ID (безкоштовний — достатньо)
68. На iPhone: **Settings → Privacy & Security → Developer Mode** → увімкніть (потрібен перезапуск)

69. Запустіть на пристрої

Альтернатива (XR Simulation): Якщо немає телефону, тестуйте в редакторі та зробіть скріншоти. Експоруйте Unity Package.

5. ІНДИВІДУАЛЬНІ ВАРІАНТИ

Кожен студент виконує **одне завдання** відповідно до свого варіанта. Базовий код наведено вище (та в папці `code/lab_02/`) — потрібно **модифікувати або доповнити** його.

5.1. Визначення вашого варіанту

Формулу обчислення, таблицю підгруп та приклади розрахунків див. у файлі **ВАРІАНТИ.md**.

5.2. Таблиця модифікаторів (Лабораторна №2)

Модифікатор визначає з **яким примітивом** з Частини 1 працює ваш варіант:

Модифікатор	Примітив	У сцені
A	Cube	червоний, (-4, 0.5, 0)
B	Sphere	синій, (-1.5, 0.5, 0)
C	Capsule	жовтий, (1.5, 0.5, 0)
D	Cylinder	зелений, (4, 0.5, 0)

- Для завдань V1–5 (Spinner): модифікатор вказує, до **якого з 4-х об'єктів** сцени застосовувати ефект
- Для завдань V6–10 (Фізика): створіть **новий об'єкт** вашого примітиву з Rigidbody (замість PhysicsCube)
- Для завдань V11–15 (Тригери): використайте **ваш примітив** як TriggerBall

5.3. Таблиця завдань (Лабораторна №2)

Важливо: індивідуальне завдання виконується в **основній сцені** (тій, що створена в Частинах 1–4). Всі базові об'єкти мають бути на місці.

Для варіантів V1–5: скрипт `Spinner.cs` висить на **всіх 4-х об'єктах**. Щоб ефект працював лише на вашому — додайте у `Spinner.cs` поле:

```
[SerializeField] private bool isVariantTarget = false;
```

У Inspector увімкніть цей прапорець **тільки** на вашому об'єкті. Весь код варіанту обгорніть в `if (isVariantTarget) { ... }`.

V	Завдання	Модифікувати	Підказка
1	Space/тап змінює напрямок обертання вашого об'єкта на протилежний. У Console — поточний напрямок	Spinner.cs	<code>speed = -speed; + Debug.Log + Input</code>
2	Швидкість обертання вашого об'єкта змінюється за синусоїдою (0→max→0). Решта обертаються з фіксованою швидкістю — для порівняння	Spinner.cs	<code>Mathf.Abs(Mathf.Sin(Time.time)) * maxSpeed</code>
3	Space/тап — обертання вашого об'єкта зупиняється і він збільшується на 20% . Повторний тап — повертає обертання та розмір	Spinner.cs	<code>bool isPaused + transform.localScale</code>
4	Колір матеріалу вашого об'єкта плавно змінюється під час обертання (rainbow). Решта зберігають свої кольори	Spinner.cs	<code>Color.HSVToRGB(Time.time % 1f, 1, 1) + GetComponent<Renderer>().material.color</code>
5	Ваш об'єкт обертається навколо сусіднього по орбіті (радіус 2). Сусідній продовжує крутитись на місці	Spinner.cs	Додайте поле [SerializeField] Transform center; — в Inspector перетягніть сусідній об'єкт. В Update: <code>transform.RotateAround(center.position, Vector3.up, speed * Time.deltaTime)</code>
6	Space/тап — ваш фізичний об'єкт стрибає . Подвійний Space/тап (2 за 0.3с)	PhysicsDemo.cs	<code>startPosition = transform.position</code> в Start(). Поле <code>float lastTapTime</code> . При тапі: якщо

	— скидає на початкову позицію		<pre>Time.time - lastTapTime < 0.3f → transform.position = startPosition; ta rb.velocity = Vector3.zero;, інакше → AddForce(Vector3.u p * forceAmount, ForceMode.Impulse) . Завжди: lastTapTime = Time.time</pre>
7	Додайте Physics Material (Bounciness=0.8). При кожному відскоку від підлоги — об'єкт змінює колір випадково	PhysicsDemo.cs	<pre>Physics Material + OnCollisionEnter + Random.ColorHSV(.. .)</pre>
8	Утримування Space/тапу заряджає силу (час у Console). При відпусканні — сила застосовується пропорційно часу утримування	PhysicsDemo.cs	<pre>float chargeTime. B Update: якщо Input.GetKey(KeyCo de.Space) або (Input.touchCount > 0 i touch.phase == TouchPhase.Moved або touch.phase == TouchPhase.Station ary) → chargeTime += Time.deltaTime + Debug.Log(chargeTi me). Якщо Input.GetKeyUp(Key Code.Space) або TouchPhase.Ended → AddForce(Vector3.u p * chargeTime * forceAmount, ForceMode.Impulse) + chargeTime = 0</pre>
9	Об'єкт обертається Torque при тапі. Під час руху кожену секунду залишає	PhysicsDemo.cs	<pre>Тап → rb.AddTorque(Vecto r3.up * forceAmount, ForceMode.Impulse)</pre>

	маркер (маленький Sphere, scale 0.1)		. Таймер: float timer В Update: timer += Time.deltaTime; if (timer >= 1f) { timer = 0; var m = GameObject.CreatePrimitive(PrimitiveType.Sphere); m.transform.localScale = Vector3.one * 0.1f; m.transform.position = transform.position; ; Destroy(m.GetComponent<Collider>()); }
10	Додайте другий об'єкт того ж примітиву (mass=10). При зіткненні — обидва змінюють колір і другий отримує силу вгору	PhysicsDemo.cs	OnCollisionEnter + other.GetComponent<Rigidbody>().AddForce(...) + GetComponent<Renderer>()
11	При вході в тригер — об'єкт отримує випадковий колір і починає обертатися . При виході — зупиняється, колір повертається	TriggerDemo.cs	bool isInTrigger → В Update(): if (isInTrigger) transform.Rotate(...) + Random.ColorHSV
12	При вході в тригер — об'єкт збільшується на 50% і moveSpeed у SimpleMovement зменшується вдвічі . При виході — навпаки	TriggerDemo.cs	Збережіть оригінальну швидкість: originalSpeed = GetComponent<SimpleMovement>().moveSpeed В Start(). Enter: moveSpeed = originalSpeed / 2f, Exit: moveSpeed = originalSpeed
13	При вході в тригер — звуковий сигнал і TriggerZone мигає 3 рази (Renderer вмикається/вимикається з інтервалом 0.2с)	TriggerDemo.cs	1) Поле [SerializeField] AudioClip clip; → у Inspector перетягніть .wav. 2) В OnTriggerEnter:

			<pre> AudioSource.PlayClipAtPoint(clip, transform.position) + zoneRenderer = other.GetComponent <Renderer>() + toggleCount = 0 + InvokeRepeating("ToggleZone", 0, 0.2f). 3) Метод void ToggleZone() { zoneRenderer.enabled = !zoneRenderer.enabled; toggleCount++; if (toggleCount >= 6) CancelInvoke("ToggleZone"); }. Увага: НЕ SetActive — інакше тригер зламається </pre>
14	При вході в тригер — створюється копія об'єкта у випадковій позиції (поза тригером!), знищується через 3с. У Console — лічильник створених	TriggerDemo.cs	<pre> var clone = Instantiate(gameOb ject, randomPos, ...) + видаліть TriggerDemo з клона: Destroy(clone.GetC omponent<TriggerDe mo>()) + Destroy(clone, 3f) + spawnCount++ </pre>
15	Лічильник входів у тригер на UI Text (TextMeshPro). При 5 входах — TriggerZone зникає назавжди	TriggerDemo.cs	1) Hierarchy → UI → Text - TextMeshPro (з'явиться Canvas + EventSystem — не видаляйте). 2) Якщо вікно "TMP Importer" — натисніть Import TMP Essentials. 3) У скрипті: using TMPro; + [SerializeField] TMP_Text counterText; 4) У Inspector перетягніть Text (TMP) у поле counterText

Наприклад, варіант **7-D** означає: створити **Cylinder** з Rigidbody + Physics Material (Bounciness=0.8), при відскоку — зміна кольору.

Варіант **3-B** означає: Space/тап зупиняє обертання **Sphere** (синьої) та збільшує її на 20%.

6. КОНТРОЛЬНІ ЗАПИТАННЯ

1. Що таке GameObject та Component в Unity? Як вони пов'язані?
2. Назвіть та поясніть три основні методи життєвого циклу MonoBehaviour.
3. Навіщо потрібен Time.deltaTime? Що станеться без нього?
4. Яка різниця між Update() та FixedUpdate()?
5. Чим відрізняється Collider від Trigger?
6. Назвіть чотири режими ForceMode та поясніть різницю між ними.
7. Що робить метод GetComponent<T>()? Чому його краще викликати в Awake(), а не в Update()?

7. ВИМОГИ ДО ЗДАЧІ

7.1. Що потрібно здати:

1. Демонстрація працюючого додатку:
 - Базова сцена з обертанням (Spinner)
 - Фізика (PhysicsDemo)
 - Тригери (TriggerDemo)
 - Індивідуальне завдання за варіантом
2. Звіт (у довільній формі, 2–3 сторінки):
 - Мета роботи
 - Скріншоти кожної частини (мінімум 4)
 - Код індивідуального завдання з поясненням
 - Посилання на відео демонстрації (30–60 сек, YouTube / Google Drive)

- Висновки

8. КРИТЕРІЇ ОЦІНЮВАННЯ

Критерій	Бали
Сцена з 4 примітивами та матеріалами	1
Spinner працює коректно	2
PhysicsDemo працює (Rigidbody + сили)	2
TriggerDemo працює (вхід/вихід)	2
Індивідуальне завдання виконано	2
Відповіді на контрольні запитання	1
Всього	10

Мобільна збірка (APK/IPA) не обов'язкова. Якщо немає телефону — демонструйте у Unity Editor та додайте скріншоти та відео до звіту.

9. РЕКОМЕНДОВАНІ ДЖЕРЕЛА

1. **Unity Scripting Reference:** docs.unity3d.com/ScriptReference
2. **Unity Learn — Scripting:** learn.unity.com
3. **Книга:** Wijesooriya I. *Mastering Augmented Reality Development with Unity*. BPB Publications, 2023. — Chapter 4–5.
4. **Книга:** Linowes J. *Augmented Reality with Unity AR Foundation*. Packt, 2021. — Chapter 2.